Problem statement

The goal is not to build a streaming platform. The goal is to build a lightweight synchronization layer that lets two people play the same movie on their own devices while the app only coordinates playback actions such as play, pause, and seek. The media file stays local on each device, and the app's job is only to keep the two playback sessions aligned through room-based messaging.

That framing matters because it makes the product smaller, safer, and much easier to build than a true video-streaming service.

Solution

Build a Windows background app that connects two users into a room and synchronizes playback commands across their local media players. The app does not render video itself and does not transmit video data. It listens for local playback events and forwards simple control events to the other side. On Windows, this can be built around the system media transport layer, because Microsoft's GlobalSystemMediaTransportControlsSessionManager provides access to playback sessions that have integrated with System Media Transport Controls, and it can also identify the current session the system believes the user most likely wants to control. (Microsoft Learn)

Why this approach

This is the simplest version of "watch together" that still feels like a real product. It avoids video hosting, avoids bandwidth-heavy streaming, avoids codec handling, and avoids the complexity of keeping two video decoders perfectly in sync. Instead, it uses the operating system's media session layer, which is specifically designed for remote control and playback-state visibility. (Microsoft Learn)

It also fits the user experience you want: a small utility that sits in the background or in the tray, rather than a full-window app that replaces the user's media player. Windows Forms' NotifyIcon component is explicitly described as a way to display icons for background processes that do not show a user interface much of the time. (Microsoft Learn)

Why it works

The app works because the only shared state that needs to travel between devices is the playback intent, not the media itself. A play event, pause event, or seek event is tiny, fast to send, and easy to apply on the other machine. For real-time message delivery, ASP.NET Core

SignalR is designed to push content from server to clients instantly, which makes it a good fit for room-based control events. (Microsoft Learn)

If you later want to remove the central relay entirely, WebRTC supports peer-to-peer exchange of generic data between peers, but for an MVP the simplest path is usually a small signaling and relay layer. (MDN Web Docs)

Demand

This kind of software has real demand because people already use watch-party behavior informally: couples, long-distance friends, roommates, and communities want a simple way to press play together without moving to a shared streaming subscription or a browser extension. The appeal is strongest when the software is invisible, fast to join, and does not force people to stop using the media player they already like.

The strongest use case is exactly the one you described: both users already have their own copy of the file, they are already on Windows, and they want a frictionless way to synchronize local playback.

How it should work

1) Background sync app, not media player replacement

The app should run as a small background utility with a tray icon, a room panel, and a connection status panel. Windows already supports notification-area icons for background processes, and the Win32 Shell_NotifyIcon API exists specifically to add, modify, or delete icons in the taskbar notification area. (Microsoft Learn)

The app should not decode video, render frames, or manage its own playback surface. Instead, it should observe and control the active media session exposed by the operating system.

2) Use the Windows media session layer as the control surface

The core integration point should be Global System Media Transport Controls. Microsoft documents this manager as the API that provides access to playback sessions throughout the system that have integrated with System Media Transport Controls, which is the right abstraction for a universal playback-control app. (Microsoft Learn)

This gives you a consistent way to:

- detect what is currently playing
- observe whether playback is playing or paused
- send play and pause commands
- potentially support seek where the player exposes it

The important limitation is that this is universal only for players that expose themselves through the Windows media transport layer. So the app should be designed as “best effort universal control,” not as a guarantee that every possible player on Windows will expose every control. That honesty should be built into the product spec.

3) Room-based session model

Each session should have one creator and one or more joiners.

Room state should include:

- room code
- host identity
- participant list
- current sync mode
- last command
- last command time
- optional current track metadata
- optional compatibility status for each participant

The room code is the only thing people need to join. No file upload, no account requirement for an MVP, and no media transfer.

4) Event-based synchronization, not continuous time streaming

The app should send events, not a constant stream of playback positions.

Examples:

- `PLAY`
- `PAUSE`
- `SEEK`
- `READY`
- `RESYNC_REQUEST`
- `PLAYER_CHANGED`

This matters because the app’s state is event-driven. A user presses play once, and the command is forwarded once. That keeps bandwidth tiny and the logic easy to reason about.

5) Host authority or mirrored authority

For simplicity, one side should be the authority in a room. The host sends the state changes, and the joiner mirrors them. That prevents duplicate or conflicting commands. For example:

- host presses play

- host app sends `PLAY`
- joiner app receives `PLAY`
- joiner's active media session plays

If you want both sides to be equal, the app can still behave symmetrically, but one side should win tie situations to avoid command loops.

6) Loop prevention

The app must distinguish between:

- local user action
- remote command applied locally

Without that distinction, a pause received from the other user could bounce back and forth forever. The solution is a simple flag or command origin tag.

Example logic:

- user presses pause locally
- send event with `origin = local`
- remote app receives it and applies pause
- remote app marks that pause as remote-applied and does not re-broadcast it

7) Room-state reconciliation

Every few commands, the app should compare local and remote state at a coarse level:

- playing vs paused
- approximate position
- active player name
- file title
- session ID if available

This is not for millisecond sync. It is just for recovery after network hiccups, missed events, or a player switching in the background.

8) Compatibility handling

Because the app is meant to work with any player that exposes media transport support, it should include a compatibility layer that checks:

- whether a session exists
- whether the current session can accept play/pause
- whether the session changes when the user switches players

- whether a player is inactive or unsupported

If the active player is unsupported, the app should show a clear status such as:

- “No controllable media session detected”
- “Player not exposing transport controls”
- “Waiting for supported player”

That is better than pretending the feature works when it does not.

9) Overlay and tray behavior

The UI should be tiny and non-intrusive.

Primary surfaces:

- tray icon
- small floating overlay
- quick room popup

The overlay should show:

- room code or joined room
- connected status
- current media title
- sync state
- whether the local player is detected
- a single button to leave the room

The tray menu can contain:

- Create room
- Join room
- Copy room code
- Mute notifications
- Reconnect
- Exit

Windows’ notification-area APIs and NotifyIcon component are suitable for exactly this kind of background-first utility. (Microsoft Learn)

10) Transport layer

For the first version, use a small realtime relay service. SignalR is a strong fit because Microsoft describes it as a library that simplifies adding real-time web functionality and enables server-side code to push content to clients instantly. (Microsoft Learn)

The server should do only this:

- create and manage room IDs
- authenticate or at least identify participants
- relay control events
- manage reconnects
- enforce host authority

It should not:

- store movie files
- inspect media contents
- process video
- transcode anything

11) Optional peer-to-peer mode

If you later want a more direct architecture, WebRTC can exchange generic application data peer-to-peer between clients, which makes it suitable for control messages. The WebRTC standards and API docs describe it as enabling media and generic data between peers, without requiring an intermediary for the data path itself. (MDN Web Docs)

For this product, though, a relay server is still easier to ship and debug. Peer-to-peer is a later optimization, not the first choice.

Recommended tech stack

Desktop client

Use C# and .NET.

That gives you the cleanest access to Windows integration, tray behavior, notifications, and the Windows media-session APIs. For the UI shell, WinUI 3 or WPF both work; the important part is that the app stays lightweight and can run in the background. For the tray icon, NotifyIcon is the right pattern. (Microsoft Learn)

Media control layer

Use the Windows media transport session APIs.

That is the layer that lets the app observe and control the active session exposed by the OS. (Microsoft Learn)

Realtime backend

Use ASP.NET Core SignalR.

It is built for instant message push, room-style communication, and client updates. (Microsoft Learn)

Optional P2P evolution

Use WebRTC data channels if you want to remove the relay server later. WebRTC supports arbitrary peer-to-peer data exchange. (MDN Web Docs)

Architecture

Client side

The client should contain four layers:

1. UI layer
   Room creation, join dialog, status overlay, tray menu.
2. Session layer
   Watches local media state, detects the active session, applies remote commands.
3. Sync layer
   Converts local actions into room events and applies incoming events safely.
4. Transport layer
   Maintains the SignalR or WebRTC connection.

Server side

The server should contain three modules:

1. Room manager
   Create room IDs, track members, manage host assignment.
2. Event relay
   Broadcast play/pause/seek commands to room participants.
3. Connection manager
   Handle reconnects, timeouts, and user presence.

Data model

A minimal room state can look like this:

```
{
  "roomId": "A7K4Q",
  "hostId": "user_1",
  "members": ["user_1", "user_2"],
```

```
  "state": {
    "status": "playing",
    "mediaTitle": "Movie Title",
    "position": 3264,
    "lastAction": "play",
    "lastActionBy": "user_1",
    "lastUpdatedAt": 1710000000
  }
}
```

A command event can look like this:

```
{
  "roomId": "A7K4Q",
  "type": "PAUSE",
  "source": "host",
  "timestamp": 1710000000
}
```

Event flow

Local pause:

1. user pauses the movie
2. client detects pause from the system media session
3. client sends a room event
4. server relays the event
5. remote client receives it
6. remote client applies pause to its own active session
7. remote client suppresses rebroadcast

Remote play:

1. host presses play
2. event relayed
3. joiner client receives it
4. joiner client resumes playback
5. joiner client updates local overlay state

State sync strategy

Because you do not care about frame-level sync, the synchronization logic can be simple:

- same command at roughly the same moment
- same pause/play state
- optional seek mirroring
- no tight clock alignment

This is important. You are not building a media conference product. You are building a shared-control product.

Failure handling

The app should handle:

- one user disconnecting and reconnecting
- unsupported player sessions
- room host leaving
- local player changing mid-session
- stale room state
- duplicate events
- delayed network messages

Practical recovery behavior:

- if the remote client reconnects, resend current room state
- if the active player changes, update the detected session automatically
- if the player becomes unsupported, freeze sync and show a clear warning
- if commands arrive out of order, keep the latest authoritative state

Polish features

Room join experience

Make joining as simple as entering a short room code. Add a one-click copy button for the host.

Presence

Show whether the other person is:

- connected
- currently playing
- paused
- out of sync
- using an unsupported player

Tiny overlay

A small always-on-top panel can show:

- room name
- partner name
- connection status
- current state
- one-click leave button

Reconnect automation

If the connection drops briefly, reconnect automatically and restore room membership.

Smart notifications

Use subtle tray notifications for:

- room joined
- room left
- player detected
- unsupported media session
- connection lost
- connection restored

Minimal chat or reaction panel

A small text chat or reaction strip can sit next to the controls so users do not need a second app during playback.

Ready check

Before starting playback, one side can click “Ready” so both know the other person has loaded the movie.

Compatibility indicator

Show a simple label such as:

- Supported
- Partial support
- Not detected

That reduces confusion when different players behave differently.

Persisted room preferences

Remember:

- last room used
- preferred device name
- overlay position
- whether tray notifications are enabled

File-title matching

Because the files are local and separate, you can optionally compare:

- filename
- duration
- file size
- media title

This does not need to be strict for the MVP, but it helps prevent mismatched playback.

Privacy-first behavior

The app should explicitly communicate that it never uploads the movie file and never shares the contents of the file, only playback commands and room metadata.

That makes the product easier to trust and easier to explain.

If needed, the next step can be a full technical specification with modules, event schemas, and a recommended Windows implementation structure.